

CityTech TTP Summer Bootcamp

Injection, Relationships, and Prisma

2026-06-24

Agenda

1. Review
2. Injection Attacks
3. Table Relationships
4. TypeScript and Prisma Setup
5. Group Project: Scattergories

Review

What are the most important things you remember from yesterday?

Follow Up: SQL Injection

SQL injection happens when untrusted **user input** is pasted into a query, so the input can change what the query does.

- Classic trick: typing `' OR '1'='1` into a login makes the `WHERE` always true, logging you in with no password
- Attackers can **read data they shouldn't**, **bypass logins**, or **modify and delete** data
- One of the **oldest and most common** web vulnerabilities (on the OWASP Top 10 for years)
- Root cause: the database can't tell your **code** from the user's **data**

Demo: Injection in Node.js

Let's run a small Node script that mimics SQL injection using `eval`.

```
node demos/sql_injection_demo.js
```

Code Injection: Mario Speedruns

Injection isn't only about databases. Speedrunners beat games like **Super Mario World** by injecting **arbitrary code**.

- With precise, frame-perfect **controller inputs**, they write bytes into the game's **memory**
- The console then **executes those bytes as instructions**, warping straight to the credits (or even running Snake or Pong)
- Same root idea as SQL injection: **data that gets treated as code**

Watch it in action: <https://www.youtube.com/watch?v=hB6eY73sLV0>

Injection Attacks, Generally

Injection is a whole **family** of bugs: untrusted input that gets run as code or commands.

- **SQL injection:** input runs as database queries
- **Command injection:** input runs as shell / OS commands
- **Cross-site scripting (XSS):** input runs as JavaScript in someone's browser
- **Code / template injection:** input reaches `eval` or a template engine

The fix is always the same idea: **keep data and code separate** (parameterize, validate, escape, and never `eval` user input).

Follow Up: Table Relationships

Tables connect to each other through **relationships**. Three common kinds:

- **One-to-One** (1:1)
- **One-to-Many** (1:N)
- **Many-to-Many** (M:N)

One-to-One (1:1)

Each row in table A links to **at most one** row in table B, and the other way around.

- A pairs with exactly one B
- B pairs with exactly one A
- Often used to split or extend a record into a second table

One-to-One: Sports Example

A **team** has one **head coach**, and a head coach leads exactly one team.

- The Yankees have one head coach
- That coach coaches only the Yankees
- One team, one coach: a **1:1** relationship

One-to-One: In SQL

Put a foreign key on one side and mark it **UNIQUE** :

```
CREATE TABLE coaches (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name        TEXT,  
  team_id     UUID UNIQUE REFERENCES teams(id)  
);
```

UNIQUE limits each team to **one** coach, which is what makes it 1:1 (not 1:many).

One-to-Many (1:N)

One row in table A links to **many** rows in table B, but each B links back to **one** A.

- A has many Bs
- Each B belongs to one A
- The most common relationship
- Viewed from B's side, the same relationship is called **many-to-one (N:1)**

One-to-Many: Sports Example

A **team** has many **players**, but each player is on one team.

- The Yankees have many players
- Each player belongs to a single team
- One team, many players: a **1:N** relationship

One-to-Many: In SQL

Put the foreign key on the "many" side (no `UNIQUE`):

```
CREATE TABLE players (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name        TEXT,  
  team_id     UUID REFERENCES teams(id)  
);
```

Many players can share the same `team_id`, so one team has many players.

Many-to-Many (M:N)

Rows in table A link to **many** rows in B, and rows in B link to **many** rows in A.

- A has many Bs
- B has many As
- Needs a third table to connect them

Many-to-Many: Sports Example

A **player** plays in many **games**, and each game has many players.

- One player appears in many games over a season
- One game features many players
- Many players, many games: an **M:N** relationship

Many-to-Many: In SQL

Use a **junction table** with a foreign key to each side:

```
CREATE TABLE game_players (  
  game_id    UUID REFERENCES games(id),  
  player_id  UUID REFERENCES players(id),  
  PRIMARY KEY (game_id, player_id)  
);
```

Each row links one player to one game; many rows together give the M:N relationship.

TypeScript and Compilation

TypeScript is JavaScript with **types**. Node and browsers can't run it directly, so you **compile** it to plain JavaScript first.

- You write `.ts` files; the compiler (`tsc`) turns them into `.js`
- Like compiling **C / C++** (to a binary) or **Java** (to bytecode): source in, runnable output out
- Usually `src/*.ts` compiles to `dist/*.js` , which you then run
- Payoff: the compiler catches **type errors before your code runs**

Prisma Setup: New Project

Create the project and set up Yarn (node-modules, not PnP):

```
mkdir my-project && cd my-project
yarn init -y

# use node_modules instead of Yarn PnP
echo "nodeLinker: node-modules" >> .yarnrc.yml
```

- Then add `"type": "module"` to `package.json` so Node runs your files as **ES modules**

Prisma Setup: Installing the Libraries

```
yarn add prisma @prisma/client @prisma/adapter-pg dotenv  
yarn add -D typescript @types/node
```

- **prisma**: the CLI (`init` , `migrate` , `generate` , `db pull`)
- **@prisma/client**: the typed client you import and query with
- **@prisma/adapter-pg**: connects Prisma to a PostgreSQL database
- **dotenv**: loads your `DATABASE_URL` from `.env`
- **typescript**: the compiler (`tsc`) that turns your `.ts` into runnable `.js`
- **@types/node**: type definitions for Node's built-ins

Prisma Setup: TypeScript

```
yarn tsc --init    # then edit tsconfig.json
```

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "lib": ["ES2022"],
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "verbatimModuleSyntax": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist", "prisma"]
}
```

What Is `tsconfig.json`?

`tsconfig.json` is the **configuration file for the TypeScript compiler**: it tells `tsc` how to turn your `.ts` into `.js`.

- `rootDir` / `outDir`: where the source is (`src/`) and where output goes (`dist/`)
- `target` / `module`: which JS version and module system to emit
- `strict`: turn on strict type-checking to catch more bugs
- `include` / `exclude`: which files to compile, and which to skip

`yarn tsc --init` generates a default one; you edit it to fit your project.

Prisma Setup: Initialize Prisma

```
yarn prisma init    # creates prisma/schema.prisma and .env
```

Edit `.env` with your connection string:

```
DATABASE_URL="postgresql://user:password@localhost:5432/mydb"
```

Prisma Setup: The Client Generator

Prisma 7 writes a **typed client** into your project. Configure it in `prisma/schema.prisma`:

```
generator client {  
  provider      = "prisma-client"  
  output        = "../src/generated/prisma"  
  moduleFormat  = "esm"  
  importFileExtension = "js"  
}
```

- `output` is **required**: Prisma 7 no longer writes the client to `node_modules`
- `moduleFormat = "esm"` and `importFileExtension = "js"` make the generated client import with `.js` extensions, so it runs as ES modules on Node

Prisma Setup: Generate

```
# if the database already exists:  
yarn prisma db pull      # read its tables into schema.prisma  
  
yarn prisma generate     # build the typed client
```

- `db pull` reads an **existing** database; use `prisma migrate` to **create** tables instead
- Re-run `generate` whenever the schema changes

Pull, Migrate, or Push?

The right command depends on where your schema starts:

- **Database already defined:** pull it into Prisma with `yarn prisma db pull`
- **Database empty:** define the schema in Prisma, then create the tables with `yarn prisma migrate dev`
- **Database defined, changing it via Prisma:** sync your changes with `yarn prisma db push`

`db pull` reads **from** the database; `migrate` and `db push` write **to** it.

The Script: `src/index.ts`

```
import 'dotenv/config';
import { PrismaClient } from './generated/prisma/client.js';
import { PrismaPg } from '@prisma/adapter-pg';

const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL });
const prisma = new PrismaClient({ adapter });

(async () => {
  console.log('Prisma Client connected');
  const cities = await prisma.cities.findMany();
  console.log('Cities:', cities);

  await prisma.$disconnect();
})();
```

Connects via the Postgres adapter, queries `cities`, then disconnects.

Prisma Setup: Compile and Run

Compile your TypeScript, then run the output:

```
yarn tsc          # src/*.ts -> dist/*.js  
node dist/index.js # run the compiled script
```

Tip: `yarn tsc --watch` recompiles automatically as you edit.

Using AI to Generate a Populate Script

A database is more useful with data in it. Use **AI** to draft a script that **populates** (seeds) your cities and teams.

- Give it your **schema** and a clear ask: *"write a Prisma script that inserts 5 cities and 15 teams across different sports"*
- It can generate realistic sample data quickly
- **Review, run, and verify** with a query before trusting it
- Disclose your AI use, and **trust but verify**

Group Project: Scattergories

Put the week together: build a multiplayer **Scattergories** game with an Express + Prisma API on Postgres.

- Teams of **3**, about **1.5 days**
- Players race to submit valid, unique answers for a given letter and topic
- Full spec: `project_specs/scattergories.md`